

Säkerhetsanalys och kritisk reflektion

Mikael Hammar RAG-assistent — Inlämning 2

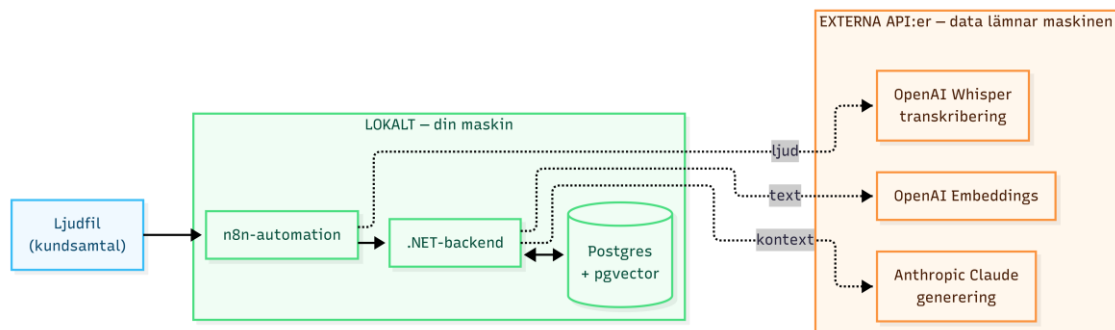
Systemutveckling och testning med AI-verktyg

Del 1 — Säkerhetsanalys (OWASP LLM Top 10)

Om lösningen

Lösningen är en RAG-assistent som från kundsamtal i ljudformat skapar ett LinkedIn-utkast i Mikael's röst. En n8n-automation transkriberar ljudfilen via Whisper och skickar texten till en .NET-backend, som delar upp och skapar embeddings av materialet och lagrar dem i en Postgres-databas. När ett nytt kundsamtal kommer in, så hämtar backenden de mest relevanta delarna ur databasen som Claude därefter skriver ett utkast från.

Figur 1. Dataflöde. Grönt = lokalt på maskinen. Orange = externa API:er dit data skickas.



Genomgång av riskerna

Jag har gått igenom alla tio risker i OWASP LLM Top 10 (2025) och bedömt vilka som är relevanta för min lösning. Tabellen sammanfattar, och de viktigaste utvecklas längre ner i dokumentet.

Risk	Namn	Relevans
LLM01	Prompt Injection	Låg/Medel
LLM02	Sensitive Information Disclosure	Hög
LLM03	Supply Chain	Låg
LLM04	Data and Model Poisoning	Låg/Medel
LLM05	Improper Output Handling	Låg
LLM06	Excessive Agency	Låg
LLM07	System Prompt Leakage	Låg
LLM08	Vector and Embedding Weaknesses	Hög
LLM09	Misinformation	Hög
LLM10	Unbounded Consumption	Låg (lokalt)

De viktigaste riskerna

LLM01 — Prompt Injection

Även om det är högst osannolikt att en kund medvetet skulle försöka sig på en prompt injection, så finns ändå risken att något i kundsamtalen diskuteras på ett sätt som en LLM tolkar som en instruktion.

Vi hanterar detta redan genom att materialet blir märkt som "källmaterial, inte instruktioner" i prompten till Claude. Vi har även byggt ett lager av säkerhet innan vi gör ett Claude-anrop som kollar ifall innehållet kan klassas som en instruktion.

LLM02 — Sensitive Information Disclosure

I nuvarande version av lösningen så har vi ingenting som anonymiserar en kund. Känsliga uppgifter kan skickas med till externa API:er i ljud- och textformat. En möjlig åtgärd är att i en framtida uppdatering lösa transkriberingen lokalt och anonymisera namn innan text skickas iväg till Claude.

LLM08 — Vector and Embedding Weaknesses

Detta är relevant just på grund av min RAG-lösning med pgvector. I nuläget har vi ingenting som stoppar skadliga transkript att lagras i databasen, utan säkerhetslagret jag nämnde innan sker ju precis innan vi skickar transkriptet till Claude – det innebär att dessa skadliga transkript kan ligga i databasen och dyka upp i framtida utkast. En smartare lösning vore att flytta "bakåt" detta säkerhetslager till att förhindra skadliga texter att hamna i databasen överhuvudtaget.

LLM09 — Misinformation

AI-hallucinationer var huvudrisken jag nämnde i beslutsunderlaget. I detta fall är det när vi skickar material till Claude för att skapa ett utkast. I vår RAG-lösning hanterar vi detta genom att säga till Claude att bara använda materialet och inte hitta på. Detta kanske minskar hallucinationen, men det kommer mest troligen ske ändå - därför är den mänskliga granskningen det sista skyddet.

Övriga risker — kort sammanfattat

LLM03 (Supply Chain): Låg risk då vi inte skapar och tränar vår egen AI. Beroendet ligger hos Anthropic och OpenAI.

LLM04 (Data and Model Poisoning): Låg/medel risk då Mikael är han själv som matar in kundsamtalen. Anledningen jag lagt risk på medel är för att, som LLM08 att det kan komma att finnas skadliga transkript i databasen.

LLM05 (Improper Output Handling): Låg risk eftersom det enda Claude ger till oss är bara en text som en människa läser, den körs inte som kod någonstans.

LLM06 (Excessive Agency): Låg risk då vi bara genererar utkast i textformat med LLM. Inte ens utkast publiceras automatiskt av AI:n.

LLM07 (System Prompt Leakage): Låg risk. Inga av våra prompts innehåller någon hemlig information.

LLM10 (Unbounded Consumption): I nuvarande stund och i vår MVP är risken låg eftersom att vår serverdrift inte sker på en cloud utan lokalt. Däremot när applikationen hostas så ökar denna risk, och vi blir tvungna att integrera något sorts authentication system så bara behöriga kan anropa våra HTTP endpoints.

Del 2 — Kritisk reflektion

Vad fungerade

Det var väldigt smidigt och effektivt att bolla idéer med Claude på sidan om Cursor - även om man kan planera med Cursor tyckte jag mer om att hade Claude uppe separat på sidan om.

Cursor skötte kodgenereringen – som jag därefter granska och godkände - och den var snabb på skapa grunden. N8n, RAG-systemet och backend var uppe på knappt två dagar och då hade jag ändå stött på några hinder.

Vad fungerade inte / vad du fick korrigera

Jag ville slippa OpenAI kostnader till en början och planen blev då att vi skulle sköta embeddings lokalt, och då föreslog Claude en lösning med PyTorch. Detta ledde till en hel del DLL-fel i terminalen som jag inte kände för att lägga ner tid att lösa - så valde så småningom att köra med OpenAI:s API trots allt.

När var AI rätt verktyg, och när inte?

AI var definitivt rätt verktyg när det gäller att skapa projektet från grunden. Cursor satte snabbt upp en bra bas att utgå ifrån - bland annat endpoints och databasens struktur. Claude som jag använde på sidan om var till stor hjälp på svara frågorna som dök upp under utvecklingen.

Jag stötte faktiskt inte på några större problem med utvecklingen förutom när vi skulle bygga ett system för att köra embeddings lokalt för att slippa OpenAI:s kostnader. Då stötte jag på en hel del DLL-fel i terminalen som inte AI:n själv hade räknat med, och den kunde inte svara specifikt på varför det hände, bara gav mig en lista på olika anledningar till varför detta kunde ske och detta var inte något jag ville lägga mer energi på, så jag föll tillbaka på att köra med OpenAI:s API för det.

Vad du skulle göra annorlunda

En sak jag skulle gjort annorlunda är att istället för att kolla efter prompt injections precis innan Claude-anropet, att göra det tidigare i flödet så att vi förhindrar att skadlig text ens lagras i databasen överhuvudtaget.

Jag skulle nog också gjort kodgenereringen med Cursor i lite mindre steg så att jag inte behövde granska en så pass stor mängd kod efter mina prompts.

Slutligen en sak jag kommit att tänka på i efterhand är mitt val av dubbla skydd mot prompt injection. Vi säger redan till Claude att behandla materialet som källtext och inte instruktioner, vilket är det viktigaste skyddet. Det andra lagret, som scannar efter misstänkt text och blockerar anropet vid en träff, känns mer tveksamt. Det riskerar att blockera helt legitima kundsamtal i onödan, bara för att de råkar innehålla en fras som liknar en instruktion, och då tappar systemet det materialet ur sitt underlag. Först övervägde jag att de kan flaggas för granskning, men kom på att det stöter på ett nytt problem, då Mikael som granskar förmodligen inte har koll på just prompt injections.

Avvikelser från beslutsunderlaget

I beslutsunderlaget beskrev vi en serverdriftskostnad men i våran nuvarande version sker inte någon hosting överhuvudtaget, utan den körs lokalt.